

PEARLS AQI PREDICTOR

PROJECT REPORT

Project Overview

This project was assigned by 10Pearls Pakistan as an end-to-end Machine Learning Systems Engineering challenge. The goal was to build a fully serverless Air Quality Index (AQI) prediction system capable of forecasting AQI for the next 3 days, using a real-world production-grade architecture consisting of a feature pipeline, a training pipeline, an automated CI/CD system, and an interactive web dashboard. The target city selected for this implementation was Karachi, Pakistan, which is among the cities with some of the most severe and volatile air quality conditions in South Asia.

The project specification outlined four major components: a feature pipeline that fetches and stores weather and air quality data, a historical backfill to generate training data, a training pipeline that selects and registers the best ML model, and a web application to serve forecasts to end users. The project brief recommended considering Hopsworks or Vertex AI as feature stores, AQICN or OpenWeather as data APIs, Streamlit or Gradio as the UI framework, and GitHub Actions or Apache Airflow for automation.

What the brief did not reveal, and what became the dominant challenge throughout this project was that nearly every recommended technology in its original form either had critical free-tier limitations, service reliability issues, version incompatibilities, or was simply incompatible with the target deployment environment. As a result, significant architectural decisions had to be made iteratively, under pressure, and with careful evaluation of alternatives at every stage.

Architecture

The final architecture is 100% serverless and composed of the following components:

Data Sources: Open-Meteo Weather Forecast API and Open-Meteo Air Quality API. Both are free, no auth required, high-uptime REST APIs that return hourly weather and pollutant readings including PM2.5, PM10, NO₂, Ozone, Temperature, Humidity, Wind Speed, Precipitation, Surface Pressure, and US AQI.

Feature Store: A DVC-tracked Apache Parquet file hosted on DagsHub Storage. The Parquet file (`data/features.parquet`) acts as a lightweight, append only feature store that grows by one row per hour. DVC manages versioning and syncing of the binary Parquet artifact, while Git only commits the lightweight `.dvc` pointer file.

Model Registry and Experiment Tracking: MLflow hosted on DagsHub. Three models are registered as `aqi_karachi_day1`, `aqi_karachi_day2`, `aqi_karachi_day3` — each promoted to a `@production` alias upon successful training.

CI/CD Automation: GitHub Actions with two workflows. The feature pipeline workflow runs on a `0 * * * *` schedule (hourly) and appends a fresh observation row to the feature store. The training pipeline workflow runs on a `0 2 * * *` schedule (daily at 2am UTC) and retrains all three horizon models.

Web Application: Gradio running on Hugging Face Spaces. The app loads production models from MLflow on startup, fetches live Karachi conditions from Open-Meteo at request time, runs inference, and renders a dark-themed dashboard with AQI gauges, 3-day forecast cards, health advisories, pollutant level cards, and a Plotly forecast trend chart.

Phase 1:

Data Source Selection and the AQICN / OpenWeather Problem

The project brief mentioned AQICN and OpenWeather as suggested data APIs. The initial approach was to use AQICN since it directly provides AQI readings and is widely cited in air quality projects.

Blocker: AQICN's free tier API token, while obtainable, returned data in a city name based lookup format that was inconsistent for Karachi. The API response structure for Pakistani cities occasionally returned null or partial station data depending on which monitoring station was active. On multiple tests calls across different times of day, the `pm2_5` and `pm10` fields came back as null because AQICN aggregates from physical monitoring stations, and Karachi's station coverage is sparse and unreliable. Building an hourly automated pipeline on top of a data source that randomly returns nulls was not viable for a production system.

Resolution: Switched to Open-Meteo, which uses atmospheric model forecasts (CAMS and ERA5) rather than physical station networks. This means coverage for any coordinate on Earth, zero authentication requirement, structured hourly JSON responses, and consistent field availability. The Open-Meteo Air Quality API provides `us_aqi`, `pm2_5`, `pm10`, `nitrogen_dioxide`, and `ozone` as hourly arrays for any latitude/longitude pair, making it ideal for a serverless automated pipeline. Open-Meteo also provides historical archive access through a separate endpoint (`archive-api.open-meteo.com`), which was critical for the backfill phase.

Phase 2:

Feature Store Selection and the Hopsworks Problem

The brief recommended Hopsworks as the primary feature store. Hopsworks is a managed MLOps platform with a Python SDK that abstracts feature groups, versioning, and retrieval. This was the first and most significant architectural blocker of the entire project.

Blocker 1: Hopsworks Server Instability: During initial setup, the Hopsworks free-tier cluster was unresponsive for extended periods. The `hopsworks.login()` call would hang indefinitely or return connection timeout errors. This happened across multiple sessions and different times of day, suggesting server-side resource saturation on the shared free tier. For a project where the feature pipeline must run on a schedule every hour without human intervention, a feature store that goes down randomly is catastrophic.

Blocker 2: Hopsworks SDK Version Conflicts: Even when the server was reachable, the hopsworks Python package introduced severe dependency conflicts. The SDK required specific versions of `hsfs` (the Hopsworks Feature Store client), which in turn conflicted with `scikit-learn`, `mlflow`, and `pandas` versions. Specifically, `hsfs` pinned `protobuf` to a range incompatible with MLflow's requirements. Attempting to resolve this via version pinning created a cascade — fixing `protobuf` broke serialization in MLflow; rolling back MLflow broke the model registry API calls that were needed for the inference layer.

Blocker 3: Hopsworks Authentication in GitHub Actions: The Hopsworks authentication model relies on a `HOPSWORKS_API_KEY` and a project-specific host URL. In GitHub Actions, environment secrets passed to the Hopsworks SDK caused additional issues because the SDK attempted to cache credentials to a local `.hopsworks` directory, and the ephemeral GitHub Actions runner filesystem didn't preserve this between steps in a reliable way, causing intermittent "unauthenticated" failures.

Resolution: After three days of debugging Hopsworks issues, the decision was made to abandon it entirely in favor of a simpler, more robust alternative. The feature store was reimplemented as a DVC-tracked Apache Parquet file stored on DagsHub. This approach has no SDK dependency, no authentication complexity in the pipeline, no server to go down, and no version conflicts. DVC handles binary versioning and remote sync via basic HTTP authentication. The Parquet file is read and written using `pandas` and `pyarrow`, which are stable and universally compatible. This turned out to be not just a workaround but arguably a better solution for the scale of this project — lightweight, fast, zero-overhead, and fully transparent.

Phase 3:

CI/CD Setup and GitHub Actions Challenges

With the data pipeline design resolved, the next phase was implementing automation using GitHub Actions.

Blocker 1: DVC Push Authentication: The first working version of the feature pipeline workflow could successfully fetch data and write the Parquet file locally, but dvc push failed with a 401 Unauthorized error even though the DAGSHUB_TOKEN secret was correctly set. The root cause was that DVC remote authentication in CI requires the credentials to be injected at runtime using dvc remote modify --local, and the --local flag writes to a .dvc/config.local file that is gitignored and not committed. In the initial workflow implementation, the remote was configured using dvc remote modify without --local, which tried to write to the committed .dvc/config and failed in the Actions environment because the working directory at that point had already been checked out as read-committed. Reordering the workflow steps, configuring the DVC remote after checkout but before the push, using --local for credentials resolved this.

Blocker 2: Git Push Permission in Actions: After the feature pipeline writes new data and DVC updates the .dvc pointer file, the workflow needs to commit and push the updated data/features.parquet.dvc file back to the repository. The initial workflow used the default GITHUB_TOKEN, but the repository's branch protection and Actions permissions settings required explicitly granting contents: write permission in the workflow YAML. Without this, the git push step failed silently or with a permissions error. This was fixed by adding permissions: contents: write at the workflow level.

Blocker 3: Dependency Installation Time: The requirements-pipeline.txt was initially identical to requirements.txt (the app requirements), which included Gradio and Plotly. These are large packages not needed for the pipeline steps. Every hourly GitHub Actions run was spending over two minutes just installing packages, consuming significant free-tier minutes. The solution was to split dependencies into two files: requirements-pipeline.txt for pipeline-only dependencies (DVC, MLflow, scikit-learn, pandas, pyarrow, requests) and requirements.txt for the full application (adding Gradio, Plotly, Hugging Face Hub). This cut CI installation time substantially.

Blocker 4: Cron Schedule Not Triggering: GitHub Actions' scheduled cron workflows are documented to run at the specified time, but in practice on free GitHub accounts, scheduled workflows can be delayed by up to one hour during periods of high runner demand. This was initially mistaken for a configuration error. After verifying the cron syntax was correct (0 * * * * for hourly), the delay was accepted as a known GitHub Actions limitation and the workflow_dispatch trigger was added to both workflows to allow manual runs during testing.

Phase 4:

Training Pipeline and MLflow Integration

Blocker 1: MLflow Model Registry API Changes: The training pipeline logs models to DagsHub's hosted MLflow instance. Between the time the initial code was written referencing the MLflow 2.x API and the requirements pinning MLflow to 3.x, breaking API changes were encountered. Specifically, `mlflow.sklearn.log_model()` in MLflow 3.x removed the `artifact_path` parameter in favor of `name`, and the return type changed so that `model_uri` was accessible directly from the `ModelInfo` object returned by `log_model()` rather than being constructed manually. The training pipeline was updated to use `name="model"` and access `info.model_uri` directly.

Blocker 2: Model Version Alias API: Setting the `@production` alias using `client.set_registered_model_alias()` requires the model to already have at least one registered version before the alias can be assigned. In early iterations, a race condition existed where the alias was being set in the same `mlflow.start_run()` context before MLflow had finished asynchronously writing the model artifact, resulting in a "model version not found" error. This was resolved by searching for model versions after `log_model()` returns and assigning the alias to the found latest version explicitly, rather than within the logging context.

Blocker 3: Insufficient Data for Training: The training pipeline has a guard that skips training if fewer than 100 rows exist in the feature store and raises a `ValueError` if fewer than 50 rows remain after applying the shift for a given forecast horizon. During development, before the backfill script had been run, the CI training workflow would fail or skip. The solution was to run `src.backfill` once manually to populate 90 days of historical hourly data (~2,160 rows) before enabling the daily training schedule. The backfill fetches from both the Open-Meteo weather archive and the air quality API for a configurable date range and writes the full dataset to the Parquet feature store.

Model Selection: Three candidate models were trained per forecast horizon: `RandomForestRegressor`, `GradientBoostingRegressor`, and a Ridge regression wrapped in a `StandardScaler` pipeline. All three were evaluated using RMSE, MAE, and R^2 . The model with the lowest RMSE on the held-out 20% test split was registered as the production version for that horizon. In practice, GradientBoosting and Random Forest consistently outperformed Ridge due to the nonlinear relationships between meteorological features and AQI, particularly the interaction between wind speed and PM2.5.

Phase 5:

Web Application and Streamlit Cloud Issues

The original plan was to deploy the web application on Streamlit Cloud, using Streamlit as the UI framework as suggested in the project brief.

Blocker 1: Streamlit Cloud and MLflow Dependency Conflict: Streamlit Cloud's build environment at the time of deployment had issues resolving the combination of mlflow, dagshub, and protobuf on Python 3.11. The dagshub package's transitive dependencies pulled in a version of protobuf that conflicted with what MLflow required for its artifact serialization. The Streamlit Cloud build would fail at the dependency resolution step with a message about protobuf version conflicts. Pinning protobuf==4.25.3 resolved the conflict locally but Streamlit Cloud's pip resolver still failed to find a satisfying set, possibly due to its own pre-installed environment packages interfering.

Blocker 2: Streamlit Cloud Secrets Handling: Even in attempts where the build succeeded, Streamlit Cloud's st.secrets system required secrets to be stored in a specific TOML format in the Streamlit Cloud dashboard. The MLflow tracking URI construction requires DAGSHUB_USERNAME and DAGSHUB_REPO to be combined at runtime. While this works fine with os.getenv(), Streamlit's secrets API doesn't automatically inject into os.environ unless the code explicitly reads from st.secrets and sets them. Adapting the entire src/Utils.py module to conditionally use either st.secrets or os.getenv() added complexity and coupling to the Streamlit framework, which was undesirable for a module that also needed to work in the GitHub Actions environment without Streamlit installed.

Resolution: Switching to Gradio on Hugging Face Spaces: The decision was made to replace Streamlit with Gradio deployed on Hugging Face Spaces. This resolved all the above issues simultaneously. Hugging Face Spaces has a well-tested build environment for MLflow and related packages, secrets are injected as standard environment variables (not a custom API), and Gradio's gr.Blocks API is sufficiently flexible to build a rich dashboard-style UI using HTML components, Plotly charts, and custom CSS. The app.py was implemented entirely in Gradio, featuring a dark glassmorphism-themed interface with an AQI gauge, color-coded 3-day forecast cards, a Plotly trend chart with AQI band shading, a health advisory alert panel, and pollutant level cards.

Blocker 3: Hugging Face Spaces Cold Start: The first request to a freshly deployed Space triggers a cold start that loads all three MLflow models from DagsHub. This can take 15–30 seconds depending on model artifact size and DagsHub response time. To mitigate this, a module-level _MODEL_CACHE dictionary was implemented in src/predict.py. Once a model is loaded for a given city/day combination, it is cached in memory and served from the cache on all subsequent requests within the same process lifetime. The demo.load() event is also wired to the same render_dashboard function so the first render fires immediately on page load rather than waiting for a user button click.

Phase 6:

EDA and Notebook Work

An exploratory data analysis notebook (notebooks/EDA.ipynb) was produced to understand the Karachi AQI dataset. The key findings were:

AQI in Karachi follows strong diurnal patterns, with peaks in early morning hours (around 6–9am) when surface temperature inversions trap pollutants near ground level, and lower values in the afternoon when convective mixing disperses particulates. Monthly seasonality is also pronounced, with winter months (November–January) showing significantly higher AQI due to temperature inversions, reduced wind, and crop burning in surrounding agricultural regions.

PM2.5 was found to be the most strongly correlated single pollutant with overall US AQI (Pearson r above 0.9), which confirmed that features derived from PM2.5 would carry the most predictive weight. Wind speed showed a strong negative correlation with AQI — higher winds disperse pollutants, lowering AQI and this relationship was nonlinear, reinforcing the choice of tree-based models over linear regressors.

The correlation heatmap confirmed that PM10 and PM2.5 are highly collinear ($r > 0.85$), as expected since both measure particulate matter at different size thresholds. NO₂ and Ozone showed a weak negative correlation with each other, consistent with photochemical dynamics. All these findings were documented as visualizations saved to the notebooks directory.

Phase 7:

Smoke Tests and Production Hardening

A smoke test suite (tests/test_runtime_imports.py) was written and wired to a GitHub Actions workflow (smoke_tests.yml) that runs on every push to the main branch. The tests verify two things: first, that all modules can be imported without requiring any environment variables to be set at import time (critical for CI environments and Hugging Face Spaces where secrets are injected post-startup), and second, that `get_city_config()` correctly raises a `ValueError` for unsupported cities and returns correct coordinates for Karachi.

This test caught a real bug: an early version of `src/utils.py` called `os.getenv("DAGSHUB_USERNAME")` and immediately used the result in a string expression at module import time to construct the MLflow URI. This caused an `AttributeError` when the environment variable was absent, meaning any import of the module in a clean environment including during the smoke test would fail. The fix was to move all environment variable reads inside the functions that need them (`get_mlflow_tracking_uri`, `get_dagshub_dvc_remote_url`) rather than at module level.

Additional production hardening included: defensive `_safe_get()` helpers in `predict.py` that handle `None` values in API response arrays; AQI clipping to the valid range `[0, 500]` after prediction to prevent negative or absurdly high forecasts from out-of-distribution inputs; duplicate timestamp elimination in the feature store append logic to prevent duplicate rows if the hourly pipeline runs twice in the same hour due to a manual trigger or retry; and a minimum data size guard in the training pipeline that skips training gracefully rather than crashing if the feature store is too small.

Tech Stack:

Final vs Originally Planned

The project brief recommended Hopsworks or Vertex AI as the feature store. The delivered system uses DVC + DagsHub Parquet storage. The brief recommended AQICN or OpenWeather as data sources. The delivered system uses Open-Meteo Weather Forecast API and Open-Meteo Air Quality API. The brief recommended Streamlit or Gradio and Flask or FastAPI for the UI. The delivered system uses Gradio on Hugging Face Spaces (no Flask/FastAPI layer needed since Gradio serves its own web server). The brief recommended GitHub Actions or Apache Airflow for CI/CD. The delivered system uses GitHub Actions with two purpose-built workflows. MLflow on DagsHub serves as the model registry and experiment tracker, which was not in the original brief but was adopted after evaluating that Hopsworks's model registry was inseparable from its broken feature store SDK.

Challenges Summary

The Hopsworks server was unreliable on the free tier and the Python SDK introduced unresolvable dependency conflicts with MLflow and scikit-learn, requiring a complete pivot to DVC + DagsHub. AQICN provided inconsistent and sometimes null data for Karachi due to sparse physical station coverage, requiring a switch to Open-Meteo's model-based forecasts. Streamlit Cloud's build environment failed to resolve protobuf version conflicts introduced by the dagshub and mlflow packages together, and its secrets system added unwanted framework coupling. GitHub Actions DVC authentication required careful step ordering and the `--local` flag for credential injection. The MLflow 3.x API introduced breaking changes from 2.x in `log_model()` parameter names and return types. Model alias assignment required careful sequencing after artifact registration to avoid race conditions. The Hugging Face Spaces cold start latency was mitigated with in-process model caching. Module-level environment variable access caused import-time failures in clean environments, caught and fixed by the smoke test suite.

Project Deliverables

The delivered system includes an end-to-end AQI prediction system for Karachi with hourly automated data ingestion, a scalable DVC-backed feature store that grows continuously, a daily automated model retraining pipeline selecting the best regressor across three forecast

horizons, an MLflow model registry with production alias management, a Gradio web application hosted on Hugging Face Spaces displaying real-time and 3-day forecasted AQI with health advisories and pollutant breakdowns, GitHub Actions CI/CD workflows automating the full pipeline, a comprehensive EDA notebook with visualizations, and a smoke test suite ensuring production import safety.